

The Labor Allocation Framework: Full Operational Treatment of the Three Modes — Direct Human, LLM-Under-Rule, Stable-Cell — and the Architectural Properties That Make Them Coherent in CKS

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, at full operational depth, the labor allocation framework introduced in the source paper's §2.3 and named as a standalone architectural commitment in the parent foundational note A1.12 — articulating the three modes by which coordination work can be performed, the three architectural properties that make the modes coherent, the labor-vs-authority distinction the framework rests on, and the mode progression dynamics by which work moves between modes without changing authority structure. This note is the first of seven labor-allocation decomposition notes (A2.68–A2.74); its contribution is the integrating frame, with each component receiving standalone treatment in subsequent notes.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's parent foundational note A1.12 commits to a labor allocation framework with three modes: direct human labor (Mode 1), LLM labor under orchestration rules (Mode 2), and stable-cell automation (Mode 3). The framework is developed in the source paper at three points (§2.3, §6.3, §7.4) and named as a single architectural commitment in A1.12. This note formalizes the framework at the next level of operational depth: it specifies each mode at integrating-frame resolution, identifies the three architectural properties without which the modes cannot be jointly coherent, isolates the labor-vs-authority distinction as the framework's load-bearing derivation, and describes the mode progression dynamics. The contribution is the integrating frame; deep treatment of each mode (A2.69–A2.71), of the three architectural properties as a unified specification (A2.72), of the labor-vs-authority distinction (A2.73), and of mode progression dynamics (A2.74) is deferred to the specialization notes the integrating frame organizes.

1. Why the labor allocation framework needs operational treatment standalone

A1.12 commits to the labor allocation framework as architectural — establishing the three modes as one property of CKS: that all three are available, that authority is preserved across them, that work can move between them. What A1.12 expressly defers to subsequent decomposition is specification of the framework at full operational depth: the integrating frame at the level of detail downstream readers need to determine whether a candidate system instantiates the framework, with each mode and each architectural property

specifiable independently.

The motivating cases are deployments allocating coordination labor across humans and LLMs at scale. Without precise specification of the three modes, deployments drift in two directions. They over-engineer for human-only labor — treating CKS as requiring all coordination work to be human-performed, which fails to scale and contradicts the source paper's §6.3 cost analysis — or they under-engineer for autonomous AI, treating LLMs as having governance authority, which violates the human-governed commitment per A1.01. The integrating-frame treatment makes the framework's structure operationally visible so that drift in either direction is identifiable as drift.

A second motivation is strategic prior-art posture. The labor allocation framework forecloses architectures that conflate labor with authority in either direction; patentable derivations addressing human-AI collaboration, mixed-labor AI systems, or labor-allocation patterns are more defensibly contested when the three-mode framework is publicly formalized at operational depth.

A third motivation is the connection to A1.01 and A1.11. A1.01 commits to humans holding authority through three rights at all times. A1.11 commits to non-specialist governance accessibility. A1.12 specifies that authority is independent of labor mode, which is what makes A1.11 architecturally coherent rather than dependent on the non-specialist's authoring capacity. The three commitments compose: authority is preserved across modes (A1.12); governance is accessible to non-specialists (A1.11); the underlying authority structure is human-governed (A1.01). The integrating frame is what makes the composition operationally inspectable.

2. Mode 1 — direct human labor

Mode 1 is direct human labor: humans operate directly on substrate, reading content under the inspect right per A2.01, writing through the modify right per A2.02 or the override right per A2.03, and producing substrate writes per A2.10 carrying provenance per A2.40. Cells may be involved (humans may execute cells they author), but the labor is direct — humans are the writers; their attention and judgment produce the substrate content, and the writer attribution recorded per A2.40 names the human as writer.

Mode 1 is operationally common at deployment start, when initial coordination patterns are being established and rules are not yet authored. It is also common for specific decisions that require human judgment regardless of rule support — overrides per A2.03, decisions for which the deployment has no rule, decisions where A1.01 specifies that humans retain authority directly. The distinguishing operational features are: per-element human attention at write time; cost proportional to the substrate content humans produce; writer attribution naming the human directly. A2.69 formalizes Mode 1 standalone.

3. Mode 2 — LLM labor under orchestration rules

Mode 2 is LLM labor under orchestration rules: LLMs operate within cells per A1.04 (AI-as-substrate-mediator), reading from substrate per Property A per A2.19, writing under orchestration rules per Property B per A2.20, holding no substrate-relevant state outside substrate per Property C per A2.21, exercising no authority per Property D per A2.22, and producing substrate writes recorded with attribution per Property E per A2.23. The non-determinism of LLM output is bounded by rule-governance per Guarantee B per A2.58.

Mode 2 is operationally common when the team has authored orchestration rules and the LLM operates within rule-bounded cells. The labor allocation is humans authoring rules (one of the four authorship roles per A2.66) and LLMs producing substrate content under those rules. The architectural commitment is that LLM-produced content is governed identically to human-produced content per A1.01 — the three rights apply uniformly regardless of which mode produced the content. The distinguishing operational features are: per-rule human attention at design time; per-execution LLM attention bounded by the rule; cost proportional to rule variety per A2.33 rather than to substrate size; writer attribution naming the LLM-under-rule with rule reference. A2.70 formalizes Mode 2 standalone.

4. Mode 3 — stable-cell automation

Mode 3 is stable-cell automation: cells with rules that have reached operational stability execute largely without per-execution human attention. The same architectural commitments as Mode 2 apply — cells per A1.02, AI-as-substrate-mediator per A1.04, rule-governance per A2.20 — but the human-attention pattern is different: review at coarser granularity (per-rule rather than per-execution), with stability reducing intervention frequency. Cells in Mode 3 still execute with an LLM mediator; Mode 3 is not non-LLM execution but LLM execution under rules whose stability has reduced per-execution human attention to near zero.

Mode 3 is automated, not autonomous. The distinction is load-bearing for the framework's coherence with A1.01. Stable-cell automation operates with reduced per-execution human attention, but humans retain governance authority at all times per A2.07 (the temporal property of governance). The three rights per A2.01–A2.03 apply to Mode 3 substrate writes identically to Mode 1 and Mode 2 writes; humans can override at any time, modify the cell's rule, and inspect the cell's outputs and the conditions under which they were produced. Mode 3 reduces the labor of running coordination work; it does not reduce the authority to govern its results. The distinguishing operational features are: per-rule-with-stability-monitoring human attention; cost proportional to rule stability per A2.33; writer attribution naming the cell with rule reference and LLM execution recorded. A2.71 formalizes Mode 3 standalone.

5. Three architectural properties making the modes coherent

Three architectural properties together make all three modes coherent as one framework rather than as disconnected operational alternatives. Each is the contribution of an existing foundational commitment to the framework's coherence.

The first is the **substrate-cell boundary** per A1.02. Mode 2 and Mode 3 operate through cells; cells operate over substrate; the boundary is what makes cell-based modes architecturally coherent and what distinguishes the LLM (or stable cell) doing work from the substrate carrying the result. Without it, Modes 2 and 3 would lack the architectural distinction between writer and written content, and Mode 1 would lack the architectural identity it has against the cell-based modes.

The second is **AI-as-substrate-mediator** per A1.04. Mode 2 operationalizes the LLM-as-mediator commitment specifically; the LLM operates as substrate mediator with the five properties per A2.19–A2.23. Without A1.04, Mode 2 would lack the architectural specification of how the LLM is bounded — the LLM could in principle become autonomous agent, terminal producer, or authority-bearer. Mode 3 inherits the same boundedness as a derivative of Mode 2's LLM execution under rules whose stability has matured.

The third is **linear-cost scaling** per A1.06. The cost properties of the three modes are bounded by the cost contract per A2.29–A2.34. Modes 2 and 3 release human labor at scale because governance cost is not size-proportional per A2.33. Without A1.06, they would not produce architectural cost benefits — the labor released by automation would be consumed by governance cost, and the framework's offer of mode progression toward reduced human attention would be hollow.

The three properties are jointly necessary for mode coherence. A2.72 formalizes them as a unified specification standalone.

6. The labor-vs-authority distinction

The labor-vs-authority distinction is the load-bearing architectural derivation within the framework. **Labor** is what work is done — the production of substrate content, the execution of cells, the running of operational processes. **Authority** is who has the architectural right to govern the result — to inspect, to modify, to override per A2.01–A2.03.

The distinction is that labor allocation is independent of authority structure. Labor is allocable across the three modes; authority is not allocable at all. A non-specialist who has not exercised any of the three modes still holds authority over substrate content within their authorization scope per A1.01. A specialist who has authored every cell and refined every rule still holds no more authority than the non-specialist within the non-specialist's authorization scope. Authority is an architectural property of the substrate's design; labor is a deployment choice within that design.

This is the property that makes labor mode-independence operationally coherent. Without it, mode progression would change governance structure, non-specialist governance per A1.11 would collapse into non-specialist authorship, and authority would be a function of who performed the labor — which A1.01 expressly forecloses. The distinction is what permits CKS to scale labor through Modes 2 and 3 while keeping authority architecturally intact. A2.73 formalizes this distinction standalone.

7. Mode progression dynamics

Mode progression dynamics describe how work can move between modes over time without changing authority structure.

Forward progression: **Mode 1** → **Mode 2** when humans author orchestration rules for work they have been performing directly; **Mode 2** → **Mode 3** when rules and cells reach operational stability through refinement, after which work proceeds with reduced per-execution human attention. Reverse progression: **Mode 3** → **Mode 2** when a stable cell produces unexpected results and humans return attention to per-execution review; **Mode 2** → **Mode 1** when a rule is found inadequate and humans take direct labor over until the rule is updated. Cross-mode override: humans may exercise the override right per A2.03 at any time, returning to direct labor for specific decisions regardless of which mode the work has been operating in.

The progression is driven by deployment choices, not by architectural mandate. The architectural commitment is that the progressions are operationally feasible and that authority structure is preserved across all transitions: substrate content written under any mode is governed by the same three rights, and a transition between modes changes only the writer attribution recorded per A2.40, not the authority structure under which the content stands. A2.74 formalizes mode progression dynamics standalone.

8. What the labor allocation framework does NOT claim

Stating precisely what the framework does not commit to keeps the integrating-frame treatment from drifting into something stronger than the source paper supports.

It does not require all three modes in every deployment. A deployment may operate entirely in Mode 1, entirely in Mode 2, or in any combination. The architectural commitment is that all three modes are *available*; specific deployments may exercise a subset.

It does not specify when work should move between modes. Mode progression is a deployment choice; the architectural commitment is to the progressions being operationally feasible, not to specific timing.

It does not claim mode-independence at outcome level. Different modes may produce different specific outcomes under the same conditions. The architectural commitment is to authority-mode-independence, not outcome-mode-equivalence.

It does not specify which mode is preferable. The architecture is mode-agnostic; specific deployments may have preferences based on operational concerns (cost, accuracy, throughput, audit posture), but no preference is imposed by the framework.

It does not foreclose other labor modes. Future architectural extensions may add additional modes; the three named here are the foundational modes the source paper commits to within its scope. The integrating frame is open to extension.

It does not specify a particular provenance schema. The framework requires that mode be inspectable per write, but does not commit to specific field structures, encoding schemes, or storage forms for the attribution. Provenance schema is the contribution of A1.07 and its decomposition (notably A2.40); the framework here depends on that contribution rather than supplying it.

9. Operational test at the integrating-frame level

A system instantiates the labor allocation framework at the integrated level if and only if all of the following are true at all times during the substrate's existence.

(a) The three modes are operationally available. Deployments can perform coordination work in any of the three modes per the operational tests in A2.69 (Mode 1), A2.70 (Mode 2), and A2.71 (Mode 3); no mode is foreclosed by the system's design.

(b) The three architectural properties are operationally satisfied. The substrate-cell boundary per A1.02 holds for cell-based modes; AI-as-substrate-mediator per A1.04 holds for LLM execution in Modes 2 and 3; linear-cost scaling per A1.06 holds across all three modes. A2.72 formalizes the joint test.

(c) The labor-vs-authority distinction holds operationally. For any given substrate-content element, the three rights per A2.01–A2.03 apply identically regardless of which mode's writer attribution per A1.07/A2.40 the element carries. No element type, no mode-conditional governance gate, and no mode-specific authority degradation exists in the system. A2.73 formalizes the test.

(d) Mode progression dynamics are operationally feasible. Work can move between modes per A2.74 — Mode 1 → Mode 2 by orchestration-rule authoring, Mode 2 → Mode 3 by rule stabilization, reverse progressions by attention return, override at any mode by A2.03 — without reconfiguration of governance

structure.

(e) Mode is inspectable per write. Every substrate write carries attribution sufficient for a reader exercising the inspect right per A2.01 to determine which mode produced it. The specific schema is the contribution of A1.07/A2.40; the framework's requirement is only that mode be recoverable from the attribution.

(f) The framework operates at the architectural-pattern level, not at the deployment-feature level. Specific deployments may have operational features for mode allocation (workflow tooling, mode-selection interfaces, attention dashboards), but the architectural commitment is to the underlying framework being available regardless of feature presence.

A system that fails any of (a)–(f) does not instantiate the labor allocation framework at the integrated level. The individual operational tests for each component are specified in A2.69–A2.74.

10. Why naming the integrating frame standalone matters

Implementations under pressure to deliver AI coordination value drift toward two failure patterns. Either they over-claim human-only labor — treating CKS as requiring all coordination work to be human-performed, which fails to scale and contradicts the source paper's cost analysis — or they under-claim human governance, treating LLMs as autonomous agents, which violates A1.01 and produces systems where authority structure is ambiguous or absent. The drift is steady because the precise architectural framework is operationally subtle: three modes operationally distinct, authority independent of labor, three architectural properties making modes coherent, mode progression preserving authority.

Drift away from the framework produces downstream consequences that manifest as scaling failures (human-only systems hitting direct-labor ceilings), governance failures (autonomous-AI systems violating the human-governed commitment), non-specialist governance failures (the commitment per A1.11 collapsing because governance accessibility depends on labor mode-independence), and cost-stability failures (the linear-cost commitment per A1.06 collapsing because labor released by automation is consumed by mode-conditional governance overhead).

Naming the labor allocation framework integrating frame as a standalone architectural commitment — with the three modes specified at sections 2–4, the three architectural properties at section 5, the labor-vs-authority distinction at section 6, mode progression dynamics at section 7, the limitations at section 8, and the integrating-frame operational test at section 9 — gives downstream readers a precise specification of what the framework commits to. The subsequent notes A2.69–A2.74 specialize each component; together with this integrating frame, they constitute the full operational decomposition of A1.12.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Labor Allocation Framework: Full Operational Treatment of the Three Modes — Direct Human, LLM-Under-Rule, Stable-Cell — and the Architectural Properties That Make Them Coherent in*

CKS. Derivation Note A2.68. May 5, 2026. ORCID: 0009-0004-8065-3235.